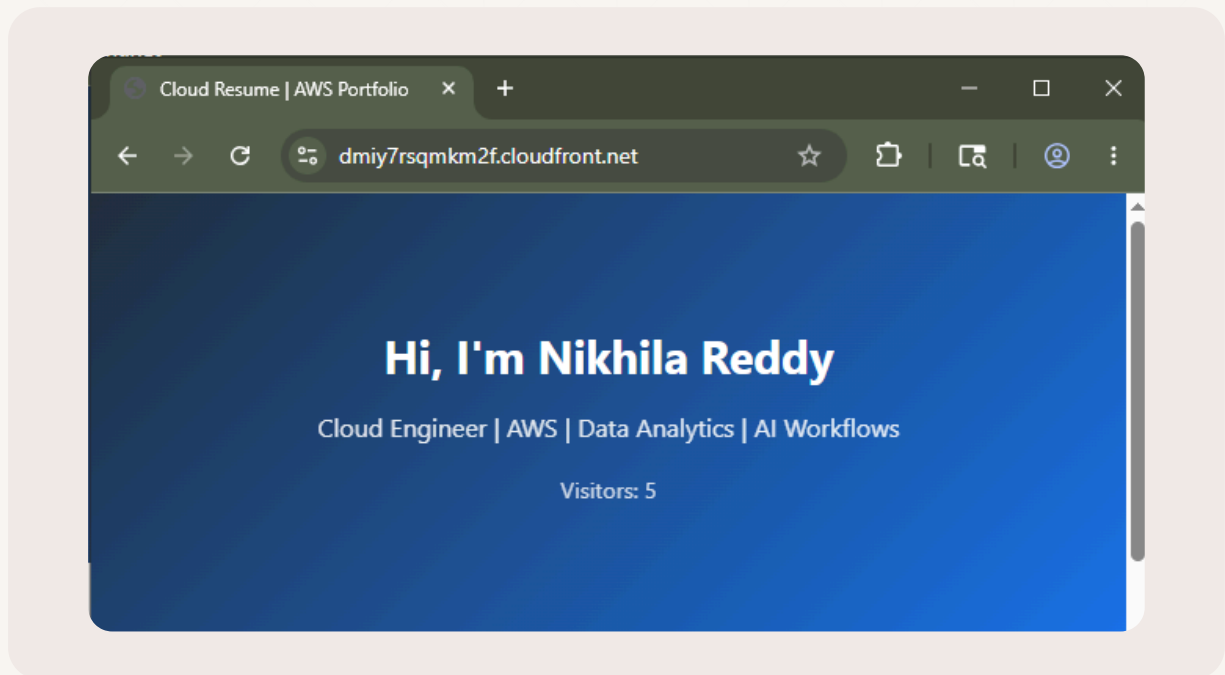


BUILD A SERVERLESS CLOUD RESUME ON AWS

By Nikhila Reddy | May 2026



Project Overview: Building a Serverless Cloud Resume on AWS

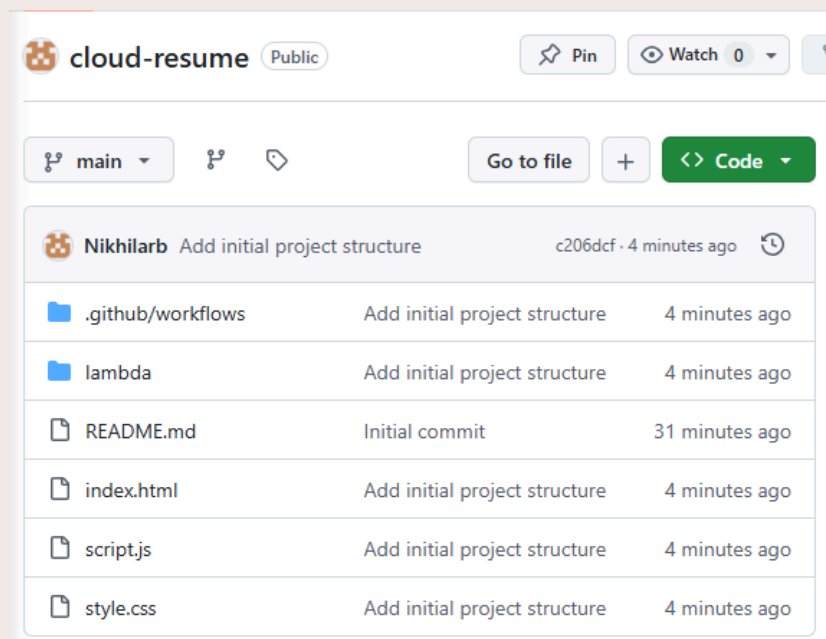
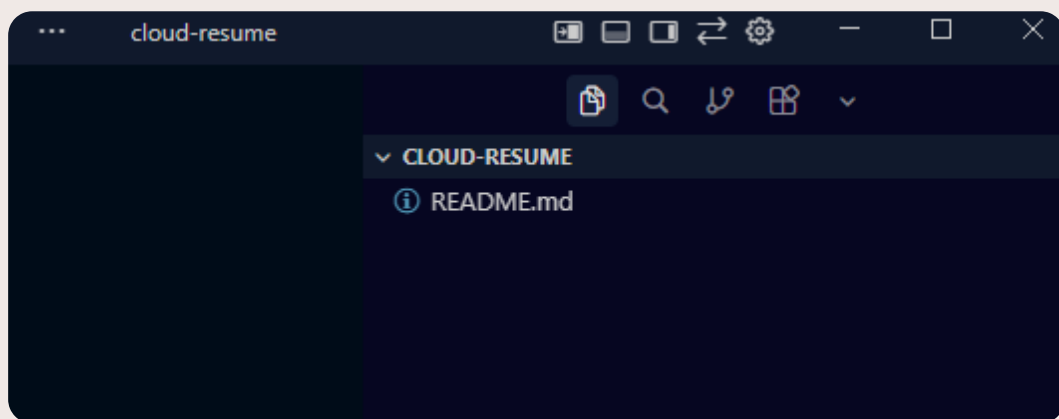
What this project sets out to achieve

In this project, I'm building a serverless cloud resume and portfolio website using AWS services like S3, CloudFront, Lambda, API Gateway, DynamoDB, and GitHub Actions so that I can learn how modern cloud applications are deployed, integrated, automated, and delivered securely at a production-style level.

Setting Up the Development Environment and Repository

Configuring Git and creating the project structure

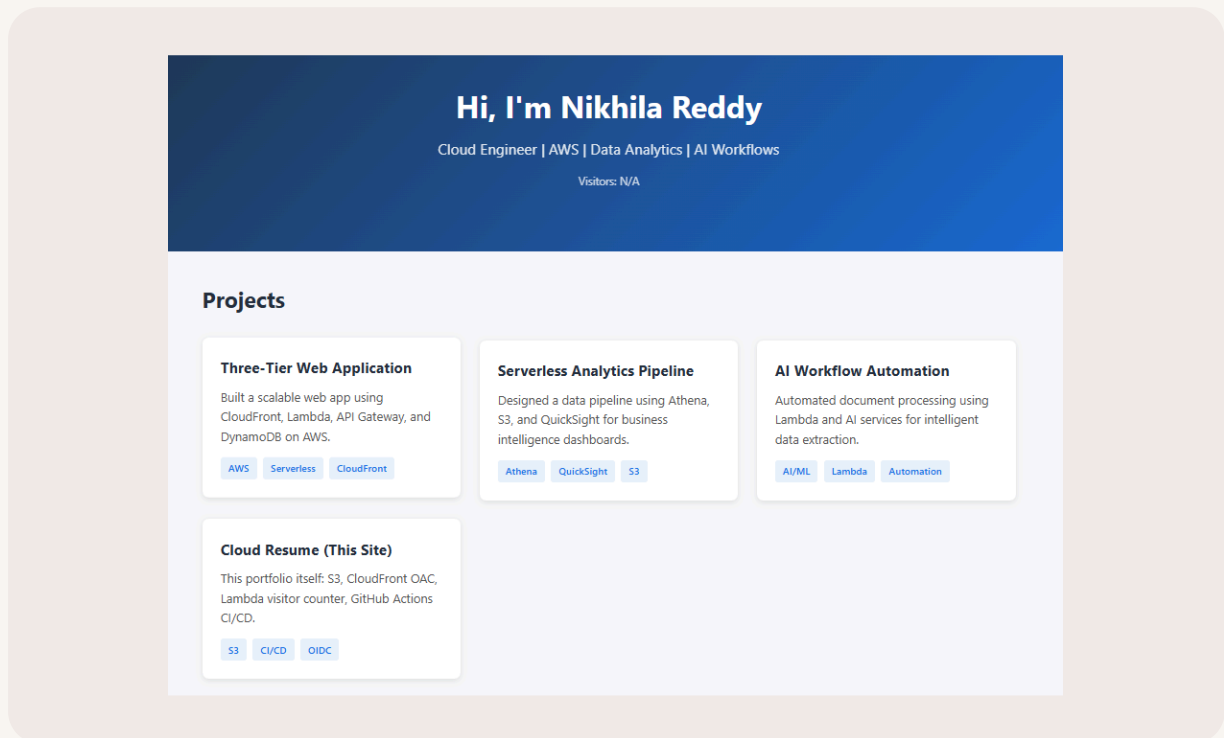
In this step, I'm setting up Git, GitHub, and my local project repository so that I can track changes, organize my project files, and enable automated deployments through a CI/CD workflow later in the project.



Building the Static Portfolio Website

Crafting the HTML, CSS, and JavaScript foundation

In this step, I'm building the frontend of my portfolio website using HTML, CSS, and JavaScript so that I can create a responsive and professional-looking cloud resume site that showcases my projects, skills, and future serverless integrations.



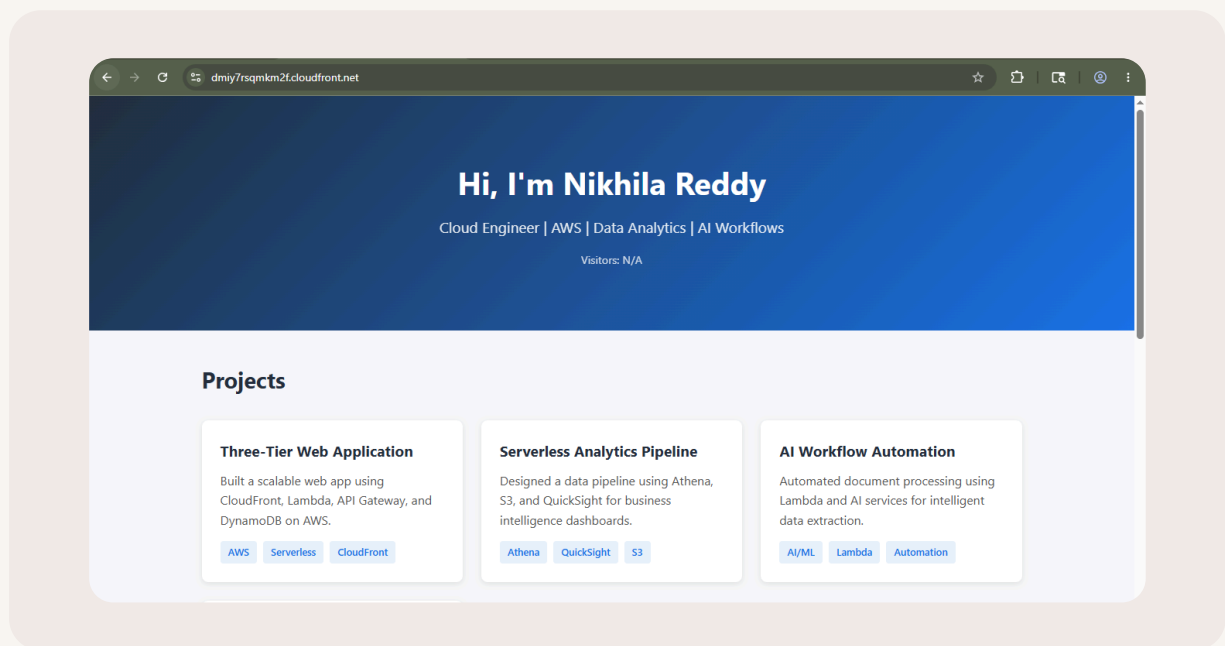
Understanding the visitor counter element

The element that holds the count is a `<span>` with the id `visitor-count`. It shows `N/A` because the JavaScript fetch call is using a placeholder API URL that doesn't exist yet, so the request fails and the catch block displays "N/A" as a fallback until the real API is built.

Deploying to S3 and CloudFront with Origin Access Control

Hosting the site on a private S3 bucket behind CloudFront

In this step, I'm deploying my site to Amazon S3 and CloudFront so that I can host my portfolio securely on AWS, deliver it globally over HTTPS, and protect the S3 bucket using Origin Access Control instead of public access.



How OAC secures the S3 origin

OAC ensures that only CloudFront can access the files in the S3 bucket while the S3 bucket stays private and blocked from direct public access.

Building the Serverless Visitor Counter Backend

Wiring together DynamoDB, Lambda, and IAM

In this step, I'm setting up a serverless backend using DynamoDB, Lambda, and IAM so that I can automatically track and update the number of visitors to my portfolio website through a secure serverless workflow.

Table: cloud-resume-visitors - Items returned (1)

 **Actions** ▾ **Create item**

Scan started on May 25, 2026, 18:15:11

< 1 > | ⚙

☐	id (String)	views
☐	visitor_count	0

☰ [Lambda](#) > [Functions](#) > CloudResumeCounter

✓ **Executing function: succeeded** ([logs](#) )

▼ **Details**

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json",
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type"
  },
  "body": "{\"count\": 2}"
}
```

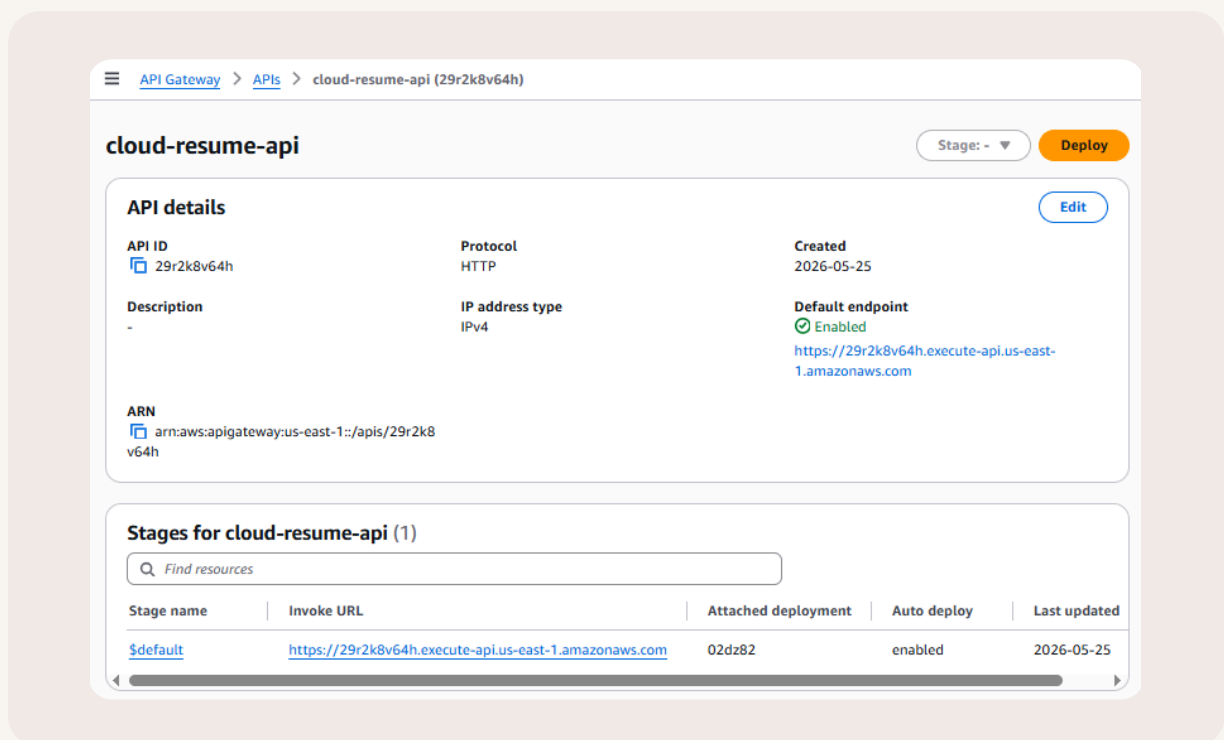
Summary

Code SHA-256 LY+IJBcF7XoKIZ+CB5T4E9VxupXIbvo0evvsyGKwrVY =	Execution time <u>1 minute ago</u>
---	--

Connecting the Frontend to the API Gateway

Exposing the counter via an HTTP API and updating the site

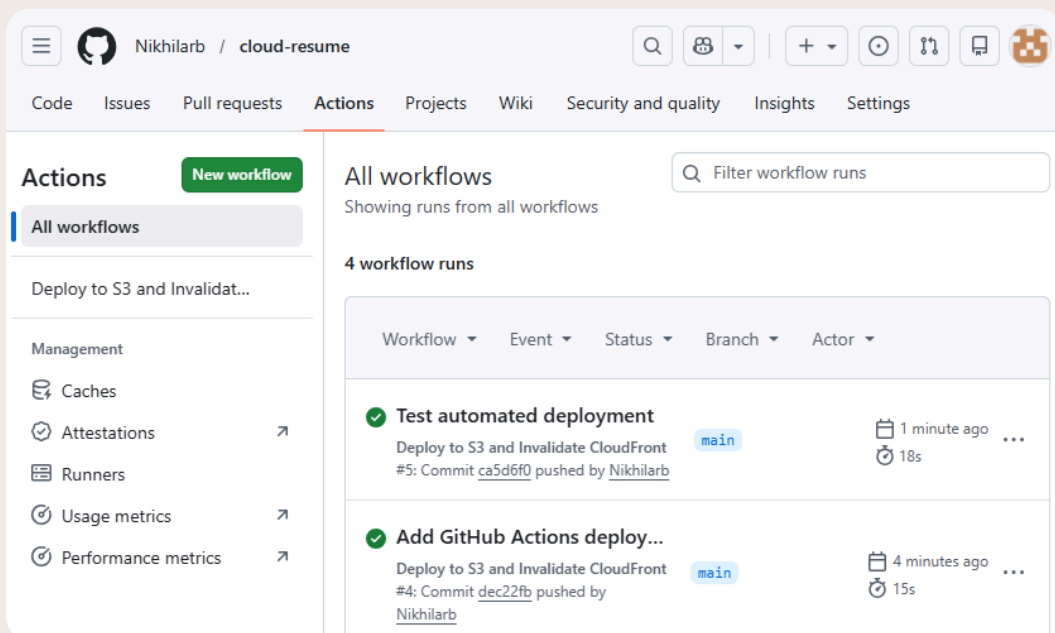
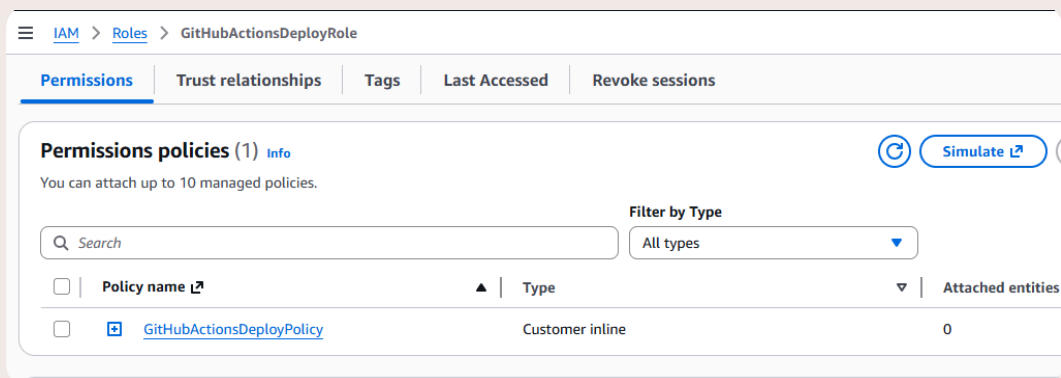
In this step, I'm creating an API Gateway HTTP API and connecting it to my Lambda function so that my portfolio website can send live HTTP requests from the browser to the serverless backend. This allows the frontend JavaScript code to retrieve and display the real-time visitor count stored in DynamoDB through a fully integrated serverless architecture.



Automating Deployments with GitHub Actions and OIDC

Setting up a secure, credential-free CI/CD pipeline

In this step, I'm setting up a CI/CD deployment pipeline using GitHub Actions and OIDC federation so that I can automatically deploy updates to my portfolio website whenever I push changes to GitHub, without manually uploading files or storing long-term AWS credentials.



Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Amazon S3, CloudFront, Lambda, API Gateway, DynamoDB, IAM, GitHub Actions, OIDC federation, Git, GitHub, HTML, CSS, and JavaScript.

Key concepts I learnt include serverless architecture, CI/CD automation, secure website hosting with Origin Access Control (OAC), API integrations, visitor counter workflows, CloudFront cache invalidation, GitHub-based deployments, and secure authentication using OIDC instead of long-term AWS credentials.

Time investment and challenges

This project took me approximately 2.5 hours including documentation. The most challenging part was repeatedly running the `aws s3 sync` command from the wrong directory while switching between CloudShell and VS Code workflows. This caused CloudShell home directory files like `.bashrc` and `.bash_history` to upload instead of the website files. I solved the issue by switching to `aws s3 cp` with full file paths, which helped me better understand deployment paths and AWS CLI workflows.

Looking ahead

I did this project today to learn how to build and deploy a production-style serverless cloud portfolio using AWS services, CI/CD workflows, APIs, and secure cloud architecture practices. Another skill I want to continue developing is designing and automating modern serverless cloud applications using AWS services and GitHub-based deployment workflows.